

Attestix: A Unified Attestation Infrastructure for Autonomous AI Agents

Pavan Kumar Dubasi
VibeTensor Private Limited
Email: pkd@vibetensor.com

Abstract—The rapid proliferation of autonomous AI agents operating across organizational boundaries introduces fundamental challenges in identity verification, trust establishment, and regulatory compliance. Existing identity and access management frameworks, designed for human users and static machine credentials, are inadequate for dynamic, capability-driven agent ecosystems. We present Attestix, an open-source attestation infrastructure that provides a unified trust layer for AI agents through nine integrated modules: cryptographic identity management, W3C Verifiable Credentials, Decentralized Identifiers (DIDs), UCAN-based delegation chains, exponential-decay reputation scoring, automated EU AI Act compliance, data provenance tracking with tamper-evident audit logs, and blockchain anchoring via the Ethereum Attestation Service. Attestix is implemented as a Model Context Protocol (MCP) server exposing 47 tools, enabling any MCP-compatible AI agent to acquire, present, and verify cryptographic attestations without modifications to the agent’s core logic. We describe the system architecture, detail the cryptographic pipeline built on Ed25519 signatures with RFC 8785 JSON canonicalization, and present evaluation results from 358 automated tests (267 functional tests including unit, integration, end-to-end, and framework compatibility suites, plus 91 conformance benchmarks) that validate standards compliance against RFC 8032 canonical vectors, W3C VC/DID specifications, and UCAN v0.9.0, demonstrating sub-millisecond cryptographic operations. Version 0.3.0 adds production integrations for LangChain, OpenAI Agents SDK, and CrewAI, a GitHub Actions CI/CD matrix, and a coordinated security-hardening batch covering delegation chain auth bypass, SSRF defences, timing-safe verification, and upstream CVE dependency pins. Attestix bridges the gap between emerging agent communication protocols (Google A2A, Anthropic MCP) and established trust standards (W3C VC, DID, UCAN), providing the first integrated compliance-aware attestation framework purpose-built for agentic AI.

Index Terms—AI agent identity, verifiable credentials, decentralized identifiers, EU AI Act, attestation, MCP, delegation, trust infrastructure

I. INTRODUCTION

The emergence of autonomous AI agents as first-class participants in digital ecosystems represents a paradigm shift in software architecture. Modern agents, powered by large language models (LLMs), independently browse the web, execute code, manage financial transactions, and coordinate with other agents across organizational boundaries [1]. Unlike traditional software services with static API keys and OAuth tokens, these agents exhibit dynamic capabilities, evolving behaviors, and multi-party delegation chains that existing identity frameworks cannot adequately represent.

This shift creates three critical gaps:

Identity Gap. When Agent A receives a request from Agent B, there is no standardized mechanism for B to prove its identity, capabilities, or authorization chain. Current approaches rely on platform-specific API keys that conflate authentication with authorization and provide no provenance information [2].

Compliance Gap. The EU Artificial Intelligence Act (Regulation 2024/1689), enforceable from August 2026, mandates risk classification, conformity assessment, technical documentation, and post-market monitoring for high-risk AI systems [5]. No existing agent framework automates these obligations at the infrastructure level.

Trust Gap. Agent-to-agent interactions lack the cryptographic verifiability that human-to-service interactions enjoy through TLS certificates and OAuth. When agents operate in open, multi-stakeholder environments, trust must be established through verifiable evidence rather than platform reputation alone [3].

We present Attestix, an open-source attestation infrastructure that addresses all three gaps through a unified architecture. Attestix operates as a Model Context Protocol (MCP) server, exposing 47 cryptographic tools across nine modules that any MCP-compatible agent can invoke. The key insight is that attestation should be a *composable infrastructure layer* rather than an application-level concern, analogous to how TLS provides transport security without requiring applications to implement cryptographic primitives.

A. Threat Model and Scope

We target an honest-but-auditable operator running the Attestix server. The trust root is the operator’s Ed25519 signing key, generated on first launch and stored with POSIX mode 00600; key compromise invalidates every downstream attestation and is treated as a catastrophic failure requiring re-issuance. We assume the following are in scope: forged UAID identities presented by a remote agent, unauthorised delegation expansion, SSRF through agent-card fetch, JSON-RPC protocol injection, silent dependency drift, replay of Verifiable Presentations, and tampering of the hash-chained audit log detected by the chain invariant. We assume the following are out of scope: side-channel attacks against the underlying Ed25519 implementation (we rely on the constant-time primitives exposed by the cryptography library at the pinned version), physical extraction of the signing key from disk, threshold or HSM-backed key storage, and post-quantum migration. Verifiers are assumed to have access to the issuer’s

DID document and to perform standard W3C Verifiable Credential verification. Attestix is an evidence-generation tool, not a legal guarantor: regulators and auditors remain the final authority on EU AI Act conformity.

B. Contributions

- A **unified attestation architecture** integrating DID-based identity, W3C Verifiable Credentials, UCAN delegation, reputation scoring, EU AI Act compliance automation, provenance tracking, and blockchain anchoring into a single coherent framework.
- A **compliance automation engine** that implements Articles 5, 9–15, 43, 72–73, and Annex V of the EU AI Act, including automated risk classification, conformity assessment enforcement, and declaration of conformity generation.
- An **open-source implementation**¹ as an MCP server with 47 tools, validated by 358 automated tests including 91 conformance benchmarks against RFC 8032, W3C VC/DID, and UCAN specifications, demonstrating that comprehensive attestation can be delivered as a protocol-level service with sub-millisecond cryptographic operations.
- A **tamper-evident audit mechanism** combining SHA-256 hash-chained logs with Merkle tree aggregation and on-chain anchoring via the Ethereum Attestation Service on Base L2 (currently deployed on Base Sepolia testnet; mainnet schema registration is planned).
- **Production framework integrations** for LangChain (BaseCallbackHandler), OpenAI Agents SDK (MCPServerStdio), and CrewAI (MCPServerAdapter), enabling any agent built on these frameworks to invoke all 47 Attestix tools without wrapper code, plus example integrations for Dify, Google ADK, Semantic Kernel, and Strands.
- A **GitHub Actions CI/CD pipeline** running the pytest matrix across Python 3.10–3.13 on Linux and Windows, with ruff, mypy, bandit, pip-audit, and safety scans gating every pull request, and a coordinated security-hardening batch remediating delegation chain authentication, SSRF, timing side-channels, and upstream dependency CVEs (PyJWT, cryptography, python-multipart).

II. BACKGROUND AND RELATED WORK

A. Agent Communication Protocols

Two recent protocols define the emerging agent interoperability landscape. The **Model Context Protocol (MCP)**, introduced by Anthropic, provides a standardized interface for LLM-based agents to discover and invoke external tools via JSON-RPC over stdio or HTTP [9]. Google’s **Agent-to-Agent (A2A) Protocol** defines agent cards, capability discovery, and inter-agent messaging through `/well-known/agent.json` endpoints [10]. Neither protocol addresses identity verification, credential exchange, or

compliance, which Attestix provides as a complementary layer.

B. Decentralized Identity Standards

The **W3C Decentralized Identifiers (DID) Core** specification [7] defines a URI scheme (`did:method:identifier`) enabling verifiable, self-sovereign identity without centralized registries. The `did:key` method encodes public keys directly in the identifier, requiring no external resolution infrastructure. The **W3C Verifiable Credentials Data Model** [6] provides a standard format for cryptographically signed claims, supporting selective disclosure and multi-party verification.

C. Authorization and Delegation

User Controlled Authorization Networks (UCAN) [8] extend the JWT format with capability-based delegation, enabling chained authorization without centralized token issuers. UCAN tokens include an `att` (attenuation) field specifying delegated capabilities and a `prf` (proof) field linking to parent tokens, forming a verifiable delegation graph.

D. AI Governance and Compliance

The **EU AI Act** (Regulation 2024/1689) establishes a risk-based regulatory framework for AI systems [5]. High-risk systems must implement risk management (Article 9), data governance (Article 10), technical documentation (Article 11), record-keeping (Article 12), transparency (Article 13), human oversight (Article 14), and accuracy guarantees (Article 15). Third-party conformity assessment is required for high-risk systems (Article 43).

E. Related Systems

Several recent works address subsets of the agent trust problem. Garzon et al. [3] propose equipping agents with DIDs and VCs for authentication, demonstrating feasibility but without compliance integration or reputation scoring. Huang et al. [2] present a zero-trust identity framework using DIDs and VCs with an Agent Naming Service, but focus on access control rather than regulatory compliance. Bodea et al. [4] address hardware-level trust through Trusted Execution Environments (TEEs), complementary to Attestix’s cryptographic approach. Chan et al. [1] survey agent infrastructure needs broadly, identifying identity binding, attribution, and interaction shaping as key functions.

Attestix differentiates itself by integrating DID-based identity, W3C Verifiable Credentials, UCAN delegation, reputation scoring, EU AI Act obligation tracking, provenance, and on-chain anchoring into a single MCP-native service. Rather than claiming novelty across every primitive, our contribution is the *composition*: to the best of our knowledge, no prior system implements the same breadth of standards in a single open-source MCP server backed by a conformance benchmark suite. Prior work either addresses subsets (Garzon [3]: DID/VC only; Huang [2]: access control; Bodea [4]: TEE; Chan [1]: survey) or stops at the design / whitepaper level. We report

¹Source code: <https://github.com/VibeTensor/attestix>

the integration itself and the conformance evidence rather than claiming performance primacy, which would require a baseline study that prior work does not yet enable.

III. SYSTEM ARCHITECTURE

Attestix follows a layered architecture with four tiers: a cryptographic foundation, a service layer implementing domain logic, a tool layer exposing MCP-compatible interfaces, and a persistence layer.

A. Module Overview

Table I summarizes the nine modules and their tool counts.

TABLE I
ATTESTIX MODULES AND TOOL DISTRIBUTION.

Module	Tools	Primary Standard
Identity	8	UAIT v0.1.0
Agent Cards	3	Google A2A Protocol
DIDs	3	W3C DID Core
Delegation	4	UCAN v0.9.0
Reputation	3	Exponential Decay
Compliance	7	EU AI Act (2024/1689)
Credentials	8	W3C VC Data Model 1.1
Provenance	5	SHA-256 Hash Chains
Blockchain	6	EAS on Base L2
Total	47	

B. Cryptographic Foundation

All operations use **Ed25519** (EdDSA over Curve25519) [11] exclusively, chosen for deterministic signatures, compact 64-byte output, the timing-attack resistance of the underlying constant-time implementation exposed by the cryptography library (pinned to $\geq 46.0.7$, see Section IV-C), and widespread DID ecosystem adoption. Attestix does not roll its own Ed25519; the timing-attack resistance claim is inherited from the upstream library, and migration to a post-quantum signature algorithm (e.g., NIST FIPS 204 ML-DSA) is listed in Section VI-B.

1) *Key Management*: The server generates a single Ed25519 signing key at first launch. When `ATTESTIX_KEY_PASSWORD` is set, the private key is encrypted using Fernet with a PBKDF2-HMAC-SHA256 derived key (480,000 iterations). The DID derived from this key serves as the server’s verifiable identity across all issued credentials.

2) *JSON Canonicalization and Signing*: Attestix implements RFC 8785 (JSON Canonicalization Scheme) for deterministic signing: (1) Unicode normalization to NFC, (2) lexicographic key sorting with compact separators, (3) whole-number float conversion per RFC 8785, (4) UTF-8 encoding, (5) Ed25519 signature encoded as `base64url`.

3) *DID:key Derivation*: Agent DIDs follow the `did:key` method:

$$\text{did:key:z||Base58btc(0xED01||pubkey}_{32}) \quad (1)$$

where `0xED01` is the Ed25519 multicodec prefix and `z` indicates base58btc multibase encoding.

C. Identity Layer

Each agent identity is represented as a **Unified Agent Identity Token (UAIT)**, a signed JSON document containing: unique identifier (`attestix:<hex16>`), display name, capabilities list, source protocol, issuer DID, temporal bounds (default 365 days), and an Ed25519 signature over immutable fields.

The signature design separates immutable creation-time fields from mutable operational fields (reputation score, compliance status, revocation), allowing updates without signature invalidation.

Identity translation bridges four formats: UAIT to A2A Agent Card, DID Document, OAuth claims, and canonical form, enabling cross-protocol interoperability.

D. Credential Layer

Credentials follow the **W3C Verifiable Credentials Data Model 1.1** with Ed25519Signature2020 proofs. The issuance pipeline: (1) construct credential with `@context`, `type`, `issuer` (server DID), and `credentialSubject`; (2) canonicalize excluding proof and `credentialStatus`; (3) sign with Ed25519; (4) attach proof with `verificationMethod` (full DID fragment URI) and `proofPurpose: assertionMethod`.

Verification checks: existence, non-revocation, non-expiry, and cryptographic signature validity.

Verifiable Presentations bundle credentials with authentication proof purpose and replay protection via `challenge/domain` fields. Five credential types are pre-defined for compliance workflows.

E. Delegation Layer

Delegation uses **UCAN v0.9.0** tokens as JWTs with EdDSA signatures, including capability attenuation (`att`), proof chains (`prf`), and cryptographic token IDs (`jti`). Chain verification enforces two invariants relevant to the v0.3.0 delegation-chain auth-bypass remediation: (i) *recursive parent validation*, where every parent token referenced in `prf` is fully verified (signature, expiry, revocation), preventing forged parent references from being admitted to the chain; and (ii) *capability subset*, where a child token’s `att` must be a subset of every ancestor’s `att`, preventing privilege escalation across the chain.

F. Compliance Layer

The compliance module automates twelve EU AI Act obligations across four risk levels. Table II maps implemented articles.

A key design decision: the system **enforces Article 43 conformity assessment requirements based on Annex III categorization**. Systems classified under Annex III Point 1 (biometric categorization) require a third-party conformity assessment; Attestix blocks `create_compliance_profile()` when the risk category falls in Point 1 and the assessment type is not `third_party`. Systems in Annex III Points 2–8 permit the internal control procedure per Article 43(2), with

TABLE II
EU AI ACT ARTICLE IMPLEMENTATION.

Article	Obligation	Mechanism
Art. 5	Prohibited practices	Rejection at creation
Art. 9	Risk management	Tracking
Art. 10	Data governance	Provenance module
Art. 11	Technical documentation	Lineage recording
Art. 12	Record-keeping	Hash-chained log
Art. 13	Transparency	Credential
Art. 14	Human oversight	Audit flag
Art. 15	Accuracy/robustness	Checklist
Art. 43	Conformity assessment	3rd-party enforced
Art. 72	Post-market monitoring	Tracking
Art. 73	Incident reporting	Tracking
Annex V	Declaration	12-field generation

a fail-safe default to third-party assessment for unknown or edge-case categorizations. The v0.3.0 security batch corrected a prior regression that applied the biometric-only rule to every high-risk category, which over-enforced third-party assessment for systems the regulation actually allows to self-assess. Gap analysis computes compliance completion as a percentage, enabling continuous monitoring.

The twelve rows above describe what Attestix *records* and, where applicable, what it *blocks* at creation or verification time. Attestix does not replace legal counsel or a notified body; deployers remain the controller for Article 16–22 obligations. A clearer breakdown of automation versus recording: Articles 5 and 43 are *enforced* at creation time (prohibited practices are rejected; biometric high-risk systems are blocked from self-assessment); Articles 10, 11, 12, 72, and 73 are *recorded* (metadata captured; obligations still require deployer process); Articles 9, 13, 14, and 15 are *tracked* (checklists, flags, audit entries that surface completion state but do not enforce it). Annex V is *generated* as a signed Verifiable Credential.

G. Reputation Layer

Trust scores use an exponential decay model:

$$S = \frac{\sum_{i=1}^n w_i \cdot e^{-\lambda \cdot (t_{now} - t_i)}}{\sum_{i=1}^n e^{-\lambda \cdot (t_{now} - t_i)}} \quad (2)$$

where $w_i \in \{0.0, 0.2, 0.5, 1.0\}$ maps to outcomes (failure, timeout, partial, success), $\lambda = \ln(2)/(30 \times 86400)$ gives a 30-day half-life, producing scores in $[0, 1]$.

H. Provenance and Audit Layer

Two mechanisms serve EU AI Act Articles 10–12:

Provenance entries: structured records for training data (Article 10) and model lineage (Article 11).

Hash-chained audit log: each entry i has a SHA-256 chain hash defined as

$$H_i = \text{SHA256}(H_{i-1} \parallel \text{JCS}(\text{entry}_i \setminus \{\text{chain_hash}, \text{signature}\})) \quad (3)$$

where H_0 is the hex string "0"⁶⁴ (genesis marker), $\parallel$ is byte concatenation, and JCS is RFC 8785 JSON Canonicalization applied to the entry minus its own `chain_hash` and `signature` fields. Each entry records action type, actor DID,

target, timestamp, input/output summaries, decision rationale, and human override flags, plus an Ed25519 signature over the same canonical bytes. A third-party verifier re-canonicalises every entry and recomputes the chain bottom-up; any tampering with any row invalidates all subsequent hashes.

I. Blockchain Anchoring

Attestix anchors artifact hashes on **Base L2** using the **Ethereum Attestation Service (EAS)**: (1) SHA-256 hash of canonical JSON, (2) EAS attestation with schema bytes32 artifactHash, string artifactType, string artifactId, string issuerDid, (3) for audit batches, Merkle root computation (RFC 6962 with domain separation) before anchoring. Integration is optional with graceful degradation.

IV. IMPLEMENTATION

Attestix is implemented in Python 3.10+ using FastMCP. The codebase comprises approximately 6,000 lines of production code across 31 files, 358 test cases (267 functional plus 91 conformance benchmarks) and example scripts for seven agent frameworks. Runtime dependencies are pinned with both lower and upper bounds to prevent silent major-version drift; key pins for v0.3.0 close upstream CVEs in PyJWT ($\geq 2.12.0$, JWT header validation bypass), cryptography ($\geq 46.0.7$, two disclosed issues), and python-multipart ($\geq 0.0.26$, DoS via multipart preamble / epilogue).

A. MCP Integration

The 47 tools are registered via module-level `register(mcp)` functions with `@mcp.tool()` decorators. `stdout` is redirected to `stderr` to prevent JSON-RPC protocol corruption.

B. Framework Integrations

Version 0.3.0 ships three production integrations that expose the full 47-tool Attestix surface to mainstream autonomous-agent frameworks without requiring per-tool wrapper code:

- **LangChain** via an `AttestixCallback` class implementing `BaseCallbackHandler`. Every tool call, LLM call, and chain step is appended to the hash-chained audit trail, and on chain completion a `ChainExecutionCredential` is auto-issued with input hash, output hash, tool sequence, and duration.
- **OpenAI Agents SDK** via `MCPServerStdio`, launched as an async context manager; the SDK auto-discovers all 47 tools, and a `create_static_tool_filter` scopes exposure per agent.
- **CrewAI** via `MCPServerAdapter`, yielding the tool list into the agent's `tools` field; role hand-offs between crew members are recorded as UCAN delegations with capability attenuation.

Four additional frameworks (Dify, Google ADK, Microsoft Semantic Kernel, Strands) consume Attestix through the generic MCP transport and ship as example configurations rather than first-party integrations.

C. Security Measures

SSRF Protection: outbound HTTP passes through an allowlist blocking private IP ranges (RFC 1918, loopback, link-local), localhost aliases, and cloud metadata endpoints (169.254.169.254, metadata.google.internal); DNS pinning validates resolved addresses before dispatch and zero redirects are permitted, closing a rebinding vector present prior to v0.3.0. **Signature Verification:** Ed25519 verification uses the constant-time implementation from `cryptography` $\geq 46.0.7$, resistant to timing attacks on the verification path. **Error Sanitization:** exception responses expose only sanitized type names; stack traces and internal file paths are never propagated to MCP or REST clients. Logging is routed to `stderr` to prevent stdout corruption of the JSON-RPC protocol. **Delegation Integrity:** the v0.3.0 security batch added recursive parent-token verification and capability-subset enforcement on every UCAN chain verification, closing a delegation-chain authentication bypass. **File Security:** on POSIX systems, the signing-key file is created with mode `0o600` (owner read/write only); key encryption uses Fernet with PBKDF2-HMAC-SHA256 (480,000 iterations) when `ATTESTIX_KEY_PASSWORD` is set. **Dependency Integrity:** runtime deps carry pinned lower and upper bounds, with lower bounds set to close upstream CVEs (PyJWT $\geq 2.12.0$, `cryptography` $\geq 46.0.7$, `python-multipart` $\geq 0.0.26$). **GDPR (technical erasure):** `purge_agent_data()` performs the technical component of Article 17 by cascading deletion across identities, credentials, compliance profiles, conformity assessments, declarations, delegations, provenance, audit logs, and reputation interactions. Article 17 also imposes organisational obligations (third-party notification, exception handling for retention duties, proof-of-erasure to the data subject) that remain the deployer’s responsibility; Attestix automates the data-plane cascade only. **Storage:** atomic writes with file locking and corruption recovery through backup rotation.

V. EVALUATION

A. Functional Correctness

The test suite includes 358 automated tests: 267 functional tests spanning unit, integration, end-to-end, and framework-compatibility suites, plus 91 conformance benchmark tests. Functional tests cover multi-agent delegation, enterprise compliance, blockchain anchoring, and persona simulations (healthcare, finance, content moderation, supply chain, cybersecurity, government regulator), as well as the three framework integrations introduced in v0.3.0. All tests are validated in clean Docker containers via `Dockerfile.test` and in the GitHub Actions CI matrix. The v0.3.0 release (2026-04-17) passes the full matrix across Python 3.10–3.13 on Ubuntu and Windows with one Windows-only skip (POSIX `chmod` assertion on the key file); the commit SHA, workflow run URL, and `requirements-lock.txt` are published with the GitHub release so that a third party can reproduce the run.

B. Standards Conformance

We validate every standards claim through dedicated conformance test suites. Table III summarizes the 91 benchmark tests.

TABLE III
CONFORMANCE BENCHMARK TEST SUITE (91 TESTS).

Standard	What is validated	Tests
RFC 8032 §7.1	4 IETF Ed25519 canonical vectors: key derivation, signature match, verify	18
W3C VC 1.1	Credential structure, proof fields, mutable field exclusion, VP, replay	24
W3C DID Core	did:key/web structure, roundtrip, Ed25519VerificationKey2020	16
UCAN v0.9.0	JWT header, payload, attenuation, expiry, revocation	16
MCP	47 tools, 9 modules, async, naming	5
Performance	Latency thresholds (see below)	7
Total conformance tests		91

RFC 8032 validation. We test against four of the five Section 7.1 test vectors (empty message, 1-byte, 2-byte, and SHA-512(abc)). For each vector, we verify: (a) public key derivation from the 32-byte seed matches the RFC-specified public key byte-for-byte, (b) the Ed25519 signature matches the RFC-specified signature exactly, (c) verification succeeds, and (d) a tampered message fails verification.

W3C VC conformance. We validate the full credential lifecycle: `@context` includes the VC v1 and Ed25519-2020 suite URIs, `type` includes `VerifiableCredential`, `proof type` is `Ed25519Signature2020` with `proofPurpose: assertionMethod`, mutable fields (`proof`, `credentialStatus`) are excluded from the signature payload, and revocation does not break the signature. VP tests validate authentication proof purpose, challenge/domain replay protection, and holder-subject binding.

W3C DID conformance. DID documents are tested for required fields (`@context`, `id`, `verificationMethod`, `authentication`, `controller`), correct verification method type (`Ed25519VerificationKey2020`), and roundtrip consistency (create then resolve).

C. Performance

Table IV reports median latencies from 1,000-iteration (cryptographic) and 100-iteration (service) benchmark runs executed inside the Docker test container.

All cryptographic primitives complete sub-millisecond. Service-layer operations include JSON file I/O and complete in low double-digit milliseconds. Blockchain latency is dominated by network round-trip and is acceptable for non-real-time compliance workflows. The performance benchmark tests enforce hard upper-bound thresholds to prevent performance regressions.

Reference hardware and reproducibility. The median latencies above are measured inside the CI runner for `pytest tests/benchmarks/` (Ubuntu x64, Python

TABLE IV
OPERATION LATENCY (MEDIAN, MEASURED IN DOCKER).

Operation	Median	Threshold
Ed25519 key generation	0.08 ms	<50 ms
JSON canonicalization (RFC 8785)	0.02 ms	<10 ms
Ed25519 sign + verify cycle	0.28 ms	<20 ms
Identity creation (UAIT)	14 ms	<200 ms
Credential issuance (W3C VC)	17 ms	<200 ms
Credential verification	2 ms	<200 ms
UCAN token creation	9 ms	<200 ms
Blockchain anchor (Base L2)	3–8 s	Network

3.12, 2 vCPU, Docker-confined, no specialised crypto hardware) and should be treated as *reference* numbers. Results on dedicated hardware with AVX-accelerated cryptography are materially faster; results on resource-constrained agents may be slower. A third party can reproduce the exact numbers by running `make bench` at the tagged v0.3.0 commit against the pinned `requirements-lock.txt` and `Dockerfile.test`. Variance is not reported: benchmark samples showed standard deviations under 15% of the median for every cryptographic primitive and under 25% for service operations during the v0.3.0 cut; we avoid over-claiming by omitting percentiles rather than reporting from a single run.

No baseline comparison. Prior work (Garzon, Huang, Bodea) does not publish comparable sign, verify, or VC-issuance latency numbers for systems we could align on workload; a formal baseline study would require re-implementing or instrumenting their prototypes, which is out of scope here. We therefore report absolute figures with thresholds rather than a comparison table.

D. Continuous Integration and Security Audits

Every push and pull request to the main branch exercises a GitHub Actions matrix across Python 3.10–3.13 on Ubuntu and Windows, running the full pytest suite (267 functional tests plus the 91-test conformance benchmark suite). Static-analysis jobs include `ruff` style and common-error checks, `mypy` type analysis, `bandit` static application security testing at medium confidence or above, `pip-audit` for transitive CVE detection, and `safety` as an advisory second opinion. All security scans were green at the v0.3.0 release (2026-04-17). The pre-release hardening batch closed a delegation chain authentication bypass, added SSRF validation on all outbound requests, moved to constant-time signature verification, sanitized API error responses, corrected Article 43 Annex III differentiation, and fixed the EAS schema UID derivation used by `anchor_credential` and `verify_anchor`.

E. Framework Integration Validation

The three v0.3.0 framework integrations (LangChain, OpenAI Agents SDK, CrewAI) are exercised end-to-end in the CI matrix. For each framework the test suite verifies: (i) Attestix identity registration produces a valid UAIT with the expected `did:key`, (ii) all 47 tools are discoverable

via the framework’s tool-invocation pathway, (iii) a representative compliance workflow (risk classification, training-data record, conformity assessment, Annex V declaration, Ed25519Signature2020 VC) executes without error, and (iv) every tool call is logged to the hash-chained audit trail with the expected actor and linkage. Example implementations for Dify, Google ADK, Semantic Kernel, and Strands are shipped as configuration-only examples.

F. Standards Coverage Summary

TABLE V
STANDARDS AND SPECIFICATIONS WITH CONFORMANCE VALIDATION.

Standard	Coverage	Validated
W3C VC Data Model 1.1	Full lifecycle	24 tests
W3C DID Core 1.0	did:key, did:web, resolver	16 tests
Ed25519 (RFC 8032)	Sign/verify/derive	18 tests
UCAN v0.9.0	Delegation chains	16 tests
RFC 8785 (JCS)	JSON canonicalization	Exercised by every VC/audit test
RFC 6962 Sec. 2.1	Merkle leaf/node hash	Unit tests
EU AI Act (2024/1689)	Enforced: Art. 5, 43 Recorded: Art. 10–12, 72–73 Tracked: Art. 9, 13–15 Generated: Annex V	E2E tests
EAS on Base Sepolia	On-chain anchoring (testnet today)	Integration + mock tests
A2A Protocol	Agent cards	Unit tests
MCP	47-tool server	5 tests

VI. DISCUSSION

A. Comparison with Related Work

TABLE VI
FEATURE COMPARISON.

Feature	Ours	[3]	[2]	[4]	[1]
DID/VC	✓	✓	✓	–	Concept
Delegation	UCAN	–	ABAC	–	Concept
EU AI Act	Full	–	–	–	–
Reputation	✓	–	–	–	–
Provenance	✓	–	–	–	Concept
Blockchain	✓	–	–	–	–
TEE	–	–	–	✓	–
MCP	47	–	–	–	–
Open Source	✓	Proto	–	–	–

B. Limitations and Future Work

Shipped in v0.3.0. Three framework integrations (LangChain, OpenAI Agents SDK, CrewAI) previously listed as future work have shipped and are exercised end-to-end in CI. A GitHub Actions CI/CD pipeline validates the full pytest and conformance suite on every push across the Python 3.10–3.13 matrix with coordinated static-analysis and security scans.

Current limitations. Single-server trust model (planned: distributed key management, threshold signing, HSM-backed keys); JSON file storage (planned: database backend); static reputation parameters (planned: adaptive Bayesian inference); no TEE integration; on-chain anchoring currently on Base

Sepolia testnet only, with mainnet schema registration deferred pending a further security review. The framework integrations are validated end-to-end in CI but have not yet been load-tested under production traffic and are best described as *reference implementations* rather than production-hardened.

EU AI Act scope boundary. The compliance layer is an evidence generator, not a legal guarantor. Attestix automates the data-plane pieces that regulators can verify (risk classification, provenance, audit trails, Annex V declaration issuance) and enforces a small number of bright-line rules (Article 5 prohibited practices, Article 43 biometric third-party assessment). Articles that require organisational process (Article 9 risk management, Article 14 human oversight, Article 72 post-market monitoring, Article 73 incident reporting) are *tracked* but still require the deployer’s own governance. Annex III categorisation is based on the deployer’s declaration of intended purpose; we do not infer the category from system behaviour.

GDPR scope boundary. Article 17 is implemented as a technical cascade only. Articles 19 (notification), 20 (data portability), 21 (objection), and 28 (processor obligations) are out of scope for v0.3.0 and listed as future work.

Cryptographic migration. Attestix is pinned to Ed25519 via the cryptography library. Post-quantum migration (NIST FIPS 204 ML-DSA, or the UCAN / VC working-group equivalents) will require schema versioning and client-side algorithm negotiation and is not yet planned.

Future work. Zero-knowledge selective disclosure; broader GDPR support beyond Article 17; DPDPA implementation for India; LGPD mapping for Brazil; NIST AI RMF and ISO/IEC 42001 controls as multi-regulatory compliance profiles; Base L2 mainnet schema registration and cross-chain anchor verification; i18n scaffolding for multi-language declarations; Trust-Over-IP ecosystem governance mappings; threshold signing and HSM-backed keys for distributed deployments.

VII. CONCLUSION

We presented Attestix, an integrated attestation infrastructure for autonomous AI agents combining cryptographic identity, verifiable credentials, capability delegation, regulatory evidence generation, reputation scoring, provenance tracking, and blockchain anchoring in a single MCP server. The system implements twelve EU AI Act articles, provides sub-millisecond cryptographic operations (0.08 ms key generation, 0.28 ms sign/verify), and bridges emerging agent protocols with established trust standards. Standards compliance is validated by 91 conformance benchmark tests covering RFC 8032 Ed25519 canonical vectors, W3C VC Data Model 1.1, W3C DID Core 1.0, and UCAN v0.9.0, all executed in Docker and in the GitHub Actions matrix alongside 267 functional tests (including framework-integration suites for LangChain, OpenAI Agents SDK, and CrewAI). Version 0.3.0 adds a co-ordinated security-hardening batch covering delegation chain authentication, SSRF, timing side-channels, and upstream dependency CVEs. Attestix is available as open-source under

Apache 2.0 at <https://github.com/VibeTensor/attestix> and on PyPI as `attestix`.

ACKNOWLEDGMENTS

This work was conducted at VibeTensor Private Limited. The author thanks the W3C, UCAN, and MCP communities for foundational standards.

REFERENCES

- [1] A. Chan, K. Wei, S. Huang, N. Rajkumar, E. Perrier, S. Lazar, G.K. Hadfield, and M. Anderljung, “Infrastructure for AI Agents,” *arXiv preprint arXiv:2501.10114*, 2025.
- [2] K. Huang *et al.*, “A Novel Zero-Trust Identity Framework for Agentic AI: Decentralized Authentication and Fine-Grained Access Control,” *arXiv preprint arXiv:2505.19301*, 2025.
- [3] S.R. Garzon, A. Vaziry, E.M. Kuzu, D.E. Gehrmann, B. Varkan, A. Gabbala, and A. Küpper, “AI Agents with Decentralized Identifiers and Verifiable Credentials,” in *Proc. 18th ICAART*, 2026.
- [4] T. Bodea *et al.*, “Trusted AI Agents in the Cloud,” *arXiv preprint arXiv:2512.05951*, 2025.
- [5] European Parliament, “Regulation (EU) 2024/1689 (Artificial Intelligence Act),” *Official Journal of the EU*, 2024.
- [6] M. Sporny, D. Longley, D. Chadwick, and O. Terbu, “Verifiable Credentials Data Model v1.1,” W3C Recommendation, 2022.
- [7] M. Sporny *et al.*, “Decentralized Identifiers (DIDs) v1.0,” W3C Recommendation, 2022.
- [8] B. Frazee, D. Holmgren, and P. Willison, “UCAN Specification v0.9.0,” 2023. [Online]. Available: <https://ucan.xyz>
- [9] Anthropic, “Model Context Protocol Specification,” 2024. [Online]. Available: <https://modelcontextprotocol.io>
- [10] Google, “Agent-to-Agent (A2A) Protocol,” 2025. [Online]. Available: <https://google.github.io/A2A/>
- [11] S. Josefsson and I. Liusvaara, “Edwards-Curve Digital Signature Algorithm (EdDSA),” RFC 8032, Internet Engineering Task Force, 2017.
- [12] P.K. Dubasi, “Attestix: Attestation Infrastructure for AI Agents,” VibeTensor, 2026. [Online]. Available: <https://github.com/VibeTensor/attestix>